TP3 - VPN Tunnel Lab

Duration: 6h

1. Lab Overview

Throughout this set of lab exercises, students will build a minimal VPN from scratch, applying the concepts of how a VPN works at a basic level. Working inside a containerized setup, they will create and configure TUN virtual interfaces, capture and inject raw IP/Ethernet packets, encapsulate IP packets inside UDP to form a tunnel between a client and a gateway, and use routing to steer traffic through the created VPN tunnel. Students will then make the tunnel bidirectional, observe what happens when it is interrupted and troubleshoot.

2. Learning Objectives

By the end of this lab, students will be able to:

- explain what a VPN is and how tunneling vs. encryption divide responsibilities in real systems;
- create a TUN interface from user space, assign an address, bring it up, and read/write IP packets through the interface;
- encapsulate IP packets inside UDP and forward them to a remote peer, forming a simple tunnel.
- configure Linux routing so selected subnets egress through the TUN device and traverse the tunnel;
- use blocking I/O with select() to multiplex between the socket and TUN file descriptors, enabling two-way traffic;
- diagnose tunnel failures using Wireshark;

3. Lab Requirements

- SEEDLabs Ubuntu 20.04 VM (22.04 for Mac);
- Internet access for downloading necessary packages.

4. Lab Setup

- To install the lab environment, follow the instructions at: SEEDS Lab Setup (If you have already done this for the first lab, skip this step);

- Download the required files for the containers you will be using throughout the assignment:
  - Labsetup-TP3.zip
  - Labsetup-arm-TP3.zip (for Mac)

5. VPN Tunnel Lab

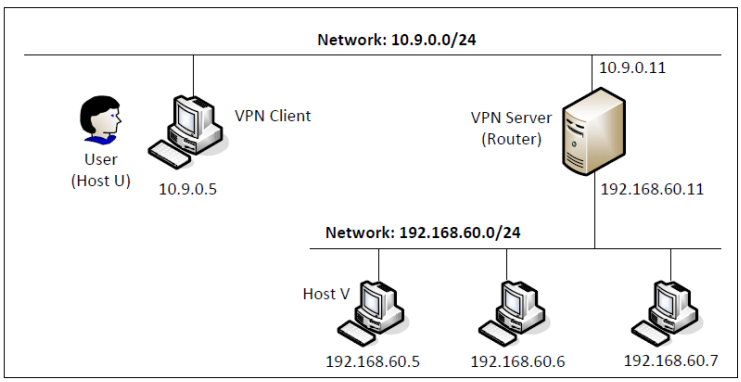


Figure 1: Lab environment setup

Task 1 - Network Setup: we will create a VPN tunnel between a computer (client) and a gateway, allowing the computer to securely access a private network via the gateway. We need at least three machines: VPN client (also serving as Host U), VPN server (the router/gateway), and a host in the private network (Host V). The network setup is depicted in Figure 1.

In a real scenario, the VPN client and VPN server are connected via the Internet. For the sake of simplicity, we directly connect these two machines to the same LAN in this lab, i.e., this LAN simulates the Internet. The third machine, Host V, is a computer inside the private network. Users on Host U (outside of the private network) want to communicate with Host V via the VPN tunnel. To simulate this setup, we connect Host V to VPN Server (also serving as a gateway). In such a setup, Host V is not directly accessible from the Internet; nor is it directly accessible from Host U.

Conduct the following tests to ensure that the lab environment is set up correctly:

- Host U can communicate with VPN Server.
- VPN Server can communicate with Host V.
- Host U should not be able to communicate with Host V.
- Run tcpdump/wireshark on the router and sniff the traffic on each of the network. Show that you can capture packets.

Task 2 - Create and Configure TUN Interface: the VPN tunnel that we are going to build is based on the TUN technology. TUN is a virtual network kernel driver; it implements network devices that are supported entirely in software. A user-space program is usually attached to the TUN virtual network interface and packets sent by an operating system via a TUN network interface are delivered to the user-space program. On the other hand, packets sent by the program via a TUN network interface are injected into the operating system network stack. To the operating system, it appears that the packets come from an external source through the virtual network interface.
When a program is attached to a TUN interface, IP packets sent by the kernel to this interface will be piped into the program. On the other hand, IP packets written to the interface by the program will be piped into the kernel, as if they came from the outside through this virtual network interface. The program can use the standard read() and write() system calls to receive packets from or send packets to the virtual interface.
The objective of this task is to get familiar with TUN technology. We will conduct several experiments to learn the technical details of the TUN interface.

Listing 1: Creating a TUN interface (tun.py)

```
#!/usr/bin/env python3

import fcntl
import struct
import os
import time
from scapy.all import *

TUNSETIFF     = 0x400454ca
IFF_TUN       = 0x0001
IFF_TAP       = 0x0002
IFF_NO_PI     = 0x1000

# Create the tun interface
tun = os.open("/dev/net/tun", os.O_RDWR)
ifr = struct.pack('16sH', b'tun%d', IFF_TUN | IFF_NO_PI)
ifname_bytes = fcntl.ioctl(tun, TUNSETIFF, ifr)

# Get the interface name
ifname = ifname_bytes.decode('UTF-8')[:16].strip("\x00")
print("Interface Name: {}".format(ifname))

while True:
        time.sleep(10)
```

Task 2.a - Name of the Interface: we will run the tun.py program on Host U. Make the above tun.py program executable and run it with root privileges.
Once the program is executed, it will block. You can go to another terminal and get a new shell on the container. Then print out all the interfaces on the machine.
You should be able to find an interface called tun0. Your job in this task is to change the tun.py program, so instead of using tun as the prefix of the interface name, use some other name. Please show your results.

Task 2.b - Set up the TUN Interface: at this point, the TUN interface is not usable, because it has not been configured yet. There are two things that we need to do before the interface can be used. First, we need to assign an IP address to it. Second, we need to bring up the interface, because the interface is still in the down state. We can use the following two commands for the configuration:

```
// Assign IP address to the interface
# ip addr add 192.168.53.99/24 dev tun0

// Bring up the interface
# ip link set dev tun0 up
```

To make life easier, you may add the following two lines of code to tun.py, so the configuration can be automatically performed by the program.

```
os.system("ip addr add 192.168.53.99/24 dev {}".format(ifname))
os.system("ip link set dev {} up".format(ifname))
```

After running the two commands above, look up your interfaces again and report your observation. How is it different from before running the configuration commands?

Task 2.c - Read from the TUN Interface: in this task, we will read from the TUN interface. Anything coming out from the TUN interface is an IP packet. We can cast the data received from the interface into a Scapy IP object, so we can print out each field of the IP packet.

Run the revised tun.py program on Host U, configure the TUN interface accordingly, and then conduct the following experiments. Describe your observations:

- On Host U, ping a host in the 192.168.53.0/24 network. What is printed out by the tun.py program? What has happened? Why?

- On Host U, ping a host in the internal network 192.168.60.0/24. Does tun.py print out anything? Why?

Task 2.d - Write to the TUN Interface: in this task, we will write to the TUN interface. Since this is a virtual network interface, whatever is written to the interface by the application will appear in the kernel as an IP packet.
We will modify the tun.py program, so after getting a packet from the TUN interface, we construct a new packet based on the received packet. We then write the new packet to the TUN interface. How the new packet is constructed is up to you.
Modify the tun.py code according to the following requirements:

- After getting a packet from the TUN interface, if this packet is an ICMP echo request packet, construct a corresponding echo reply packet and write it to the TUN interface. Please provide evidence to show that the code works as expected.

- Instead of writing an IP packet to the interface, write some arbitrary data to the interface, and report your observation.

Task 3 - Send the IP Packet to VPN Server Through a Tunnel: in this task, we will put the IP packet received from the TUN interface into the UDP payload field of a new IP packet and send it to another computer. Namely, we place the original packet inside a new packet. This is called IP tunneling. The tunnel implementation is just standard client/server programming. It can be built on top of TCP or UDP. In this task, we will use UDP. Namely, we put an IP packet inside the payload field of a UDP packet.

We will run tun server.py program on VPN Server. This program is just a standard UDP server program. It listens to port 9090 and prints out whatever is received. The program assumes that the data in the UDP payload field is an IP packet, so it casts the payload to a Scapy IP object, and prints out the source and destination IP address of the enclosed IP packet.

Listing 2: tun server.py

```
#!/usr/bin/env python3

from scapy.all import *

IP_A = "0.0.0.0"
PORT = 9090

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind((IP_A, PORT))

while True:
        data, (ip, port) = sock.recvfrom(2048)
        print("{}:{} --> {}:{}".format(ip, port, IP_A, PORT))
        pkt = IP(data)
        print(" Inside: {} --> {}".format(pkt.src, pkt.dst))
```

Implement the client program tun client.py. First, we need to modify the TUN program tun.py. Let's rename it and call it tun_client.py. Sending data to another computer using UDP can be done using the standard socket programming.
Replace the while loop in the program with the following:

```
# Create UDP socket
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

while True:
        # Get a packet from the tun interface
        packet = os.read(tun, 2048)
        if packet:
        # Send the packet via the tunnel
                sock.sendto(packet, (SERVER_IP, SERVER_PORT))
```

The SERVER IP and SERVER PORT should be replaced with the actual IP address and port number of the server program running on VPN Server.

Run the tun server.py program on VPN Server, and then run tun client.py on Host U. To test whether the tunnel works or not, ping any IP address belonging to the 192.168.53.0/24 network. What is printed out on VPN Server? Why?

Our ultimate goal is to access the hosts inside the private network 192.168.60.0/24 using the tunnel. Let us ping Host V and see whether the ICMP packet is sent to VPN Server through the tunnel. If not, what are the problems?

Provide proof to demonstrate that when you ping an IP address in the 192.168.60.0/24 network, the ICMP packets are received by tun server.py through the tunnel.

Task 4 - Set Up the VPN Server: after tun server.py gets a packet from the tunnel, it needs to feed the packet to the kernel, so the kernel can route the packet towards its final destination. This needs to be done through a TUN interface, just like what we did in Task 2. Modify tun server.py so it can do the following:

- Create a TUN interface and configure it.
- Get the data from the socket interface; treat the received data as an IP packet.
- Write the packet to the TUN interface.

If everything is set up properly, we can ping Host V from Host U. The ICMP echo request packets should eventually arrive at Host V through the tunnel. Show proof of your work. It should be noted that although

Host V will respond to the ICMP packets, the reply will not get back to Host U because we have not set up everything yet. Therefore, for this task, it is sufficient to show (using Wireshark or tcpdump) that the ICMP packets have arrived at Host V.

Task 5 - Handling Traffic in Both Directions: after getting to this point, one direction of your tunnel is complete, i.e., we can send packets from Host U to Host V via the tunnel. If we look at the Wireshark trace on Host V, we can see that Host V has sent out the response, but the packet gets dropped somewhere. This is because our tunnel is only one directional; we need to set up its other direction, so returning traffic can be tunneled back to Host U.
To achieve that, our TUN client and server programs need to read data from two interfaces, the TUN interface and the socket interface. All these interfaces are represented by file descriptors, so we need to monitor them to see whether there is data coming from them. One way to do that is to keep polling them, and see whether there are data on each of the interfaces. The performance of this approach is undesirable, because the process has to keep running in an idle loop when there is no data. Another way is to read from an interface. By default, read is blocking, i.e., the process will be suspended if there are no data. When data become available, the process will be unblocked, and its execution will continue. This way, it does not waste CPU time when there is no data.
The read-based blocking mechanism works well for one interface. If a process is waiting on multiple interfaces, it cannot block on just one of the interfaces. It has to block on all of them altogether. Linux has a system call called select(), which allows a program to monitor multiple file descriptors simultaneously. To use select(), we need to store all the file descriptors to be monitored in a set, and then we give the set to the select() system call, which will block the process until data are available on one of the file descriptors in the set. We can check which file descriptor has received data. In the following Python code snippet, we use select() to monitor a TUN and a socket file descriptor.

```
while True:
# this will block until at least one interface is ready
        ready, _, _ = select.select([sock, tun], [], [])

for fd in ready:
        if fd is sock:
        data, (ip, port) = sock.recvfrom(2048) pkt = IP(data)
        print("From socket <==: {} --> {}".format(pkt.src, pkt.dst))
         ... (code needs to be added by students) ...

        if fd is tun:
        packet = os.read(tun, 2048) pkt = IP(packet)
        print("From tun ==>: {} --> {}".format(pkt.src, pkt.dst))
        ... (code needs to be added by students) ...
```

Once this is done, we should be able to communicate with Machine V from Machine U, and the VPN tunnel (un-encrypted) is now complete. Show proof, as well as a diagram of the flow packets now use.

Task 7 - Routing Experiment on Host V: in a real VPN system, the traffic will be encrypted (this part is not covered in this lab). That means the return traffic must come back from the same tunnel. How to get the return traffic from Host V to the VPN server is non-trivial. Our setup simplifies the situation. In our setup, Host V's routing table has a default setting: packets going to any destination, except the 192.168.60.0/24 network, will be automatically routed to the VPN server.
In the real world, Host V may be a few hops away from the VPN server, and the default routing entry may not guarantee to route the return packet back to the VPN server. Routing tables inside a private network must be set up properly to ensure that packets going to the other end of the tunnel will be routed to the VPN server. To simulate this scenario, we will remove the default entry from Host V and add a more specific entry to the routing table, so the return packets can be routed back to the VPN server.

Task 8 - VPN Between Private Networks: in this task we are setting up a VPN between two private networks. The setup is illustrated in Figure 2. The whole setup is described in the docker-compose2.yml file, and you can use the "-f docker-compose2.yml" option to ask docker-compose to use this file, instead of the default docker-compose.yml file.

This setup simulates a situation where an organization has two sites, each having a private network. The only way to connect these two networks is through the Internet. Your task is to set up a VPN between these two sites, so the communication between these two networks will go through a VPN tunnel.

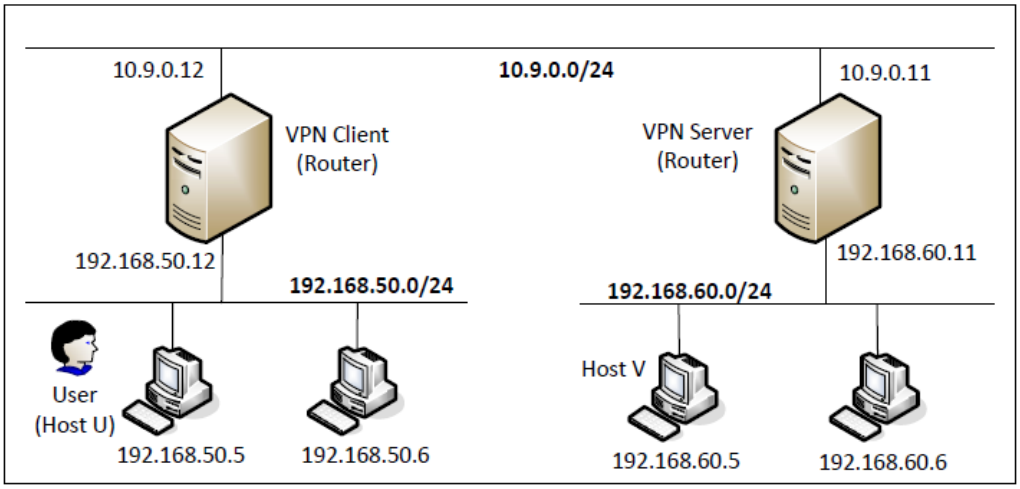


Figure 2: VPN between two private networks

6. Report and submission

Each working group is expected to:

- write a detailed report containing all answers will need to be delivered by the deadline for this assignment. Responses should be clear and include enough information through text and/or screen captures that proves you understood the concept;
- provide answers and/or code that was clearly written by AI will not be accepted. Those exercises will be marked with a 0;
- indicate which question you are answering, using the same order as stated in the PDF assignment;
- write a brief conclusion with your learning experiences, as well as any point where you particularly struggled;
- complete the submission through the BlackBoard Assignment hand-in. Each working group should upload a single .zip file containing the report and code files. It should follow this naming scheme: NS-TPx-PL1y.zip, where x is the TP number and y is the working group ID. The report and code files should follow the same naming scheme: NS-TPx-PL1y.pdf, NS-TPx-PL1y.c and NS-TPx-PL1y.py.